

Problem Set 4

Harvard SEAS - Fall 2026

Due: Mon Oct. 19, 2026 (11:59pm)

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments.

If you did not have any collaborators or external resources, please write 'none.' A problem set on the border between two letter grades cannot be rounded up if collaborators are not noted.

Your name:

Collaborators and External Resources:

No. of late days used on previous psets:

No. of late days used after including this pset:

The purpose of this problem set is to solidify your understanding of the RAM model (and variants), and the relations between the RAM model, the Word-RAM model, Python programs, and variants. In particular, you will build skills in simulating one computational model by another and in evaluating the runtime of the simulations (both in theory and in practice).

1. (Simulation in practice: RAMs on Python) In the Github repository, we have given you a partially written Python implementation of a universal RAM Model simulator. Your task is to fill in the missing parts of the code to obtain a complete universal RAM simulator. Your simulator should take as input a RAM Program P and an input x , and simulate the execution of P on x , and return whatever output P produces (if it halts). The RAM Program P is given as a Python list $[v, C_0, C_1, \dots, C_{\ell-1}]$, where v is the number of variables used by P . For simplicity, we assume that the variables are named $0, \dots, v-1$ (rather than having names like “tmpptr” and “insertvalue”), but you can introduce constants to give names to the variables. The 0^{th} variable will always be `input_len`, the 1^{st} variable `output_ptr`, and the 2^{nd} variable `output_len`. A command C is given in the form of a list of the form `[cmd]`, `[cmd, i]`, `[cmd, i, j]`, or `[cmd, i, j, k]`, where `cmd` is the name of the command and i, j, k are the indices of the variables or line numbers used in the command. For example, the command `vari = M[varj]` would be represented as `(“read”, i, j)`. See the Github repository for the precise syntax as well as some RAM programs you can use to test your simulator.

[See simulator_solution.py](#)

2. (Empirically evaluating simulation runtimes and explaining them theoretically)

Consider the following two RAM programs:

Input	: A single natural number N (as an array of length 1)
Output	: 14^{2^N+1} (as an array of length 1)
Variables	: input_len, output_ptr, output_len, counter, result
<pre> 0 zero = 0; 1 one = 1; 2 fourteen = 14; 3 output_len = 1; 4 output_ptr = 0; 5 result = 14; 6 counter = M[zero]; 7 IF counter == 0 GOTO 11; 8 result = result × result; 9 counter = counter − one; 10 IF zero == 0 GOTO 7; 11 result = result×fourteen; 12 M[output_ptr] = result; </pre>	
Input	: A single natural number N (as an array of length 1)
Output	: $14^{2^N+1} \bmod 2^{32}$ (as an array of length 1)
Variables	: input_len, output_ptr, output_len, counter, result, temp, W
<pre> 0 zero = 0; 1 one = 1; 2 fourteen = 14; 3 output_len = 1; 4 output_ptr = 0; 5 result = 14; 6 W = 2³²; 7 counter = M[zero]; 8 IF counter == 0 GOTO 15; 9 result = result × result; 10 temp = result/W; 11 temp = temp × W; 12 result = result − temp; 13 counter = counter − one; 14 IF zero == 0 GOTO 8; 15 result = result × fourteen; 16 temp = result/W; 17 temp = temp × W; 18 result = result − temp; 19 M[output_ptr] = result; </pre>	

- (a) Exactly calculate (without asymptotic notation) the RAM-model running times of the above algorithms as a function of N . Which one is faster? (4-7 sentences)

The running time for Program 1 is $4N + 10$.

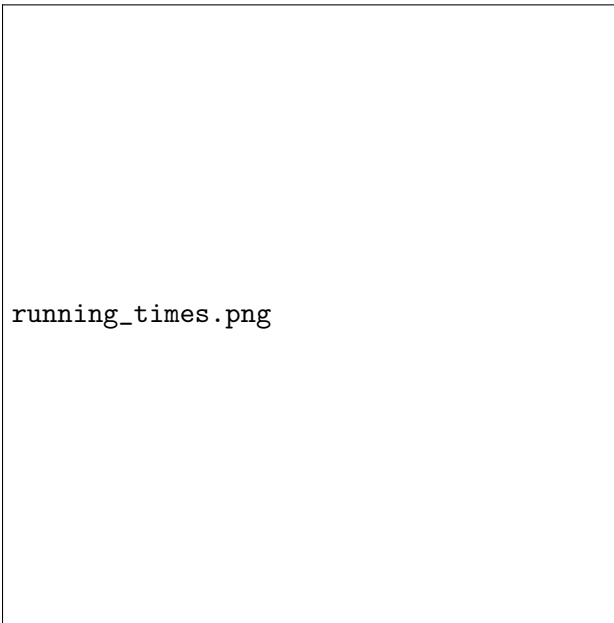
In the above code, Lines 0-6 are ran once. Then, we enter the loop from lines 7-10, decrementing the counter by one at each iteration and exiting when the counter reaches 0. Since the counter begins at N , this loop will iterate N times. We also have to check line 7 an extra time when $\text{counter} == 0$ to exit from the loop, so a total of $4N + 1$ lines will be ran in the loop. Finally, lines 11-12 will be ran once. Thus, we get a total of $7 + (4N + 1) + 2 = 4N + 10$ total lines executed.

The running time for Program 2 is $7N + 14$.

Lines 0-7 will be ran once and then we will enter the loop from lines 8-14. Again, we are decrementing the counter by one at each iteration and the counter began at N , so this loop will run N times. To exit the loop, we run line 8 a final time to check that the counter is equal to zero and then jump to the last five lines of the program, lines 15-19, which are ran once. Thus, the program executes $8 + (7N + 1) + 5 = 7N + 14$ total lines.

Since $4N + 10 < 7N + 14$ for all non-negative N , Program 1 runs faster.

- (b) Using your RAM Simulator, run both RAM programs on inputs $N = 0, 1, 2, \dots, 15$ and graph the actual running times (in clock time, not RAM steps). (We have provided you with some timing and graphing code in the Github repository.) Which one is faster? (*1-2 sentences*)



From the above graph, we can see that Program 1 grows exponentially and Program 2 runs linearly, so Program 2 is faster in general. Specifically, Program 1 is faster for $N < 12$, and Program 2 is faster for $N \geq 12$.

- (c) *In no more than 10 sentences*, explain the discrepancies you see between Parts 2a and 2b. (Hint: What do we know about the relationship between the RAM and Word-RAM models, and why is it relevant to how efficiently the Python simulation runs?)

We observe that Program 2 runs faster in practice, despite having a higher running time in part (a). We can explain this discrepancy by noticing an unfounded assumption we make in the RAM model.

In the RAM model, we assume that each line takes $O(1)$ time to execute, giving us the running times from part (a). However, in practice, the numbers in our implementation are growing very rapidly, and arithmetic operations with large numbers take more time than the same operations with small numbers. For example, the grade-school approach to multiplication runs in time $O(n^2)$ for n -digit numbers. Modern programming languages like Python may use better algorithms and number representations to improve on these times ([info about Python's bignum representation here](#)). Regardless, these operations are still dependent on input size n and not constant-time.

In Program 2, the values of our variables are bounded via the modulus operations. Since we take the $\text{mod } 2^{32}$ of the result at every iteration of the loop, the values stored in our variables will remain below 2^{64} at all times (since we multiply the result by itself in Line 9). In Program 1, there is no bound on the size of the variables, so the runtime increases much more dramatically. Altogether, while we expect both Program 1 and 2 to run in linear time from part (a), only Program 2 appears to adhere to this linear projection in practice because the time of operations depends on the size of the operands, and only Program 2 bounds the size of the operands.

- (d) (optional¹) Give a theoretical explanation of the shapes of the runtime curves you see in Part 2b, by providing explicit formulas for the asymptotic runtimes of the two programs (in clock time). You may need to do some research online and/or make guesses about how Python operations are implemented to come up with your estimates.

Out of all the operations that we are running, multiplication will take the longest time. Doing some research about how Python does multiplication ([details here](#)), Python will use the grade-school algorithm for multiplication for small numbers, running in time $O(n^2)$, where n is the maximum number of bits in the factors. For larger numbers, namely exceeding 70 digits, Python will switch to bignum arithmetic and will use Karatsuba algorithm for multiplication, which has a running time of $O(n^{\log_2 3}) \approx O(n^{1.585})$.

In Program 2, the variables being multiplied are bounded at 2^{32} , so these variables will fit in a single word and the time for multiplication will be constant. This explains why program 2 has a linear shape in the running time graphs.

Program 1, however, does not bound the variables and the variables will take up multiple words. In order to calculate the total complexity of the runtimes, we calculate the runtime of iteration i , where i ranges from 0 to $N - 1$. At the i 'th iteration, we multiply 17^{2^i} by itself. Python represents these numbers in base 2^{30} - to get the number of digits, we take the logarithm to get $\log_{2^{30}}(17^{2^i}) = 2^i \log_{2^{30}}(17)$. We then input this into the Karatsuba's algorithm runtime to get $O((2^i \log_{2^{30}}(17))^{\log_2 3}) = O((2^i)^{\log_2 3}) =$

¹This problem won't make a difference between N, L, R-, and R grades.

$O(3^i)$. Finally, we sum the runtime of a single iteration over all N iterations to get $O(3^0 + 3^1 + 3^2 + \dots + 3^{N-1}) = O(\frac{3^N - 1}{3 - 1}) = O(3^N)$. This explains the runtime graph - while the $\log_{230}(17)$ constant we ignored earlier is actually very small, for large enough values of N the exponential runtime starts to show.

Another approach notes that the final multiplication takes $O(3^N)$ and therefore bounds all N multiplications with the runtime $O(N3^N)$ - this is another correct upper bound, though it doesn't use the above mathematical analysis which allows us to tighten the bound.

3. (Simulating Word-RAM by RAM) For every Word-RAM program P , there is a RAM program R that simulates P in the following sense. For every $n \in \mathbb{N}$, input array x of length n , word-length w , and time bound t , $R(x, 2^w, \max_i x[i], t)$ should do the following:
 - If $P[w](x)$ halts without crashing within t steps, then R should halt with the same as the output of $P[w](x)$.
 - If $P[w](x)$ crashes or fails to halt within its first t steps, then the R should indicate so by halting with `output_ptr = output_len =  $2^w$` .
 - The runtime of R should be $O(t)$ and the largest memory location accessed by R should be at most $n + t$.
 - All of the values that R 's variables and memory cells hold during the computation should have bitlength at most $O(w + \log \max_i x[i] + \log n)$.

Your proof should use an *implementation-level* description, similar to the proof that RAM programs can be simulated by ones with at most c registers in Theorem 6.4 from the Hesterberg-Vadhan textbook. *Please keep your response within 1.5 pages.*

Recall that Word-RAM programs have a finite but changing memory size S ; you may want to start your simulation by initializing S . Then think about how each operation of a Word-RAM program P can be simulated in a RAM program R , taking care of any differences between their semantics in the Word-RAM model vs. the RAM model. Don't forget `MALLOC`!

Your proof should use an *implementation-level* description, similar to the proof that RAM programs can be simulated by ones with at most c registers in Theorem 6.4 from the Hesterberg-Vadhan textbook. Your answer should include - at least - the following aspects:

- Initialization: How the memory in the RAM program R will be initialized? Be mindful of the additional "default variables" that a Word-RAM program has, in comparison to the RAM program. Recall that Word-RAM programs have a finite but changing memory size S ; you may want to start your simulation by initializing S .
- Operations: For each operation in the Word-RAM program P , how will the corresponding operation in the RAM program R be performed? Keep in mind that the RAM program "may not by default know" of nuances of the arithmetic in Word-RAM.
- Memory: How will the read and write into the memory locations take place? Keep in mind that the RAM program "may not by default know" of nuances of memory reads or writes in the Word-RAM.

- Output: How will the output of the Word-RAM program P be simulated in R ? Again, keep the differences between Word-RAM and RAM program in mind.
- Runtime: Justify how your simulation maintains the runtime stated above.
- Bitlength: justify how your simulation maintains the bitlength stated above.
- Don't forget MALLOC and updates to the GOTO command!

We give an implementation-level description of our RAM Program R to account for each of the differences from the RAM model.

First, note that we can perform a *comparison* between variables a and b in RAM. If $a - b == 0$, then $b \geq a$ (due to clamping of subtraction), and we can use GOTO on a variable representing their difference to go to a different code block to simulate an if statement for $b \geq a$. Then, this comparison runs in $O(1)$ time. We can further extend this argument if we wanted to have other comparisons (e.g. $<, >, \leq$, etc.), but for the following description we only use the $\geq$ operator. For the rest of the proof, we'll denote these set of commands as $a \geq b$ for concision.

Initialization: The RAM program will accept an input array x of length $n + 3$. In initialization, the first n values of x is placed into the memory locations, $(M[0], M[1], \dots, M[n - 1])$. Note that for R , `input_len` is initialized to n , where n is the length of $x - 3$. The last three values of the input array are w , which is the word-size, $\max_i x[i]$, which is the maximum value from the first n values of x , and t , the maximum step size.

At the top of R , we will add some commands to define some variables. First, let `one` = 1 and let `zero` = 0. Set `input_len` = `input_len` - 1 and $M[\text{input_len}] = 0$. Also let `S` = `input_len` + `zero`. Then set `steps` = 0.

Then, check if either `input_len` $\geq 2^w$ or $\max_i x[i] \geq 2^w$. Also check if any of the constants c in the assignment commands (`var` = c) in P are $\geq 2^w$. If so, we add a GOTO to a series of lines that sets `output_len` = 2^w and `output_ptr` = 2^w . Let us also denote the starting line for these commands as g .

Time Bound: After every line in P , set `steps` = `steps` + `one`. Also after every line, we check if `steps` = t , and if so, GOTO g .

Memory: Whenever P reads or writes from memory at locations $M[i]$ for $i \geq S$, do nothing (note that valid locations only go up to $S - 1$). The largest memory location accessed by R will be at most $n + t$ because we begin with an input of size n which we store in memory, and then we perform at most t commands, each of which store at most one more item into memory.

Operations: To simulate a line of the form `vari = varj op vark` from P in P' , we perform the same operation in P' . Due to the memory limit of Word-RAM, we check if `vari` $\geq 2^w$; if so, we set `vari` = $2^w - 1$. For a line of the form `var` = c , we'll add an additional line to check `var` $\geq 2^w$, and GOTO g if this comparison is true.

MALLOC: Increment S by 1. Then we check if $S \geq 2^w$. If yes, GOTO g .

GOTO: For each GOTO commands, after simulating the other commands the line numbers of certain instructions have changed, so we need to change the line number to reflect where the jump in the original program would have gone.

Output: Above, we've defined writes in locations larger than S to have no effect, so output will be equivalent to Word-RAM, i.e. the output is defined to be $(M[\text{output_ptr}], M[\text{output_ptr}+1], \dots, M[\min(S-1, \text{output_ptr} + \text{output_len} - 1)])$.

Runtime: Comparison run in $O(1)$ time. In initialization, adding the lines each run in $O(1)$. Checking if `input_len` or $\max_i x[i]$ is less than 2^w runs in $O(n)$. The commands added to account for the Memory, Operation, MALLOC, and GOTO differences can be added in $O(1)$ time. We only consider the first t commands from the Word-RAM program, which also each run in $O(1)$. Lastly, for each command, we also increment the step counter. Thus, in total, R will have a runtime of $O(t)$.

Bitlength: The ceiling on variable and memory values given by the word length is 2^w , which has bitlength $\log(2^w) = w$. Additionally, at initialization, each value in the input array is placed into memory (before checking that its size is under 2^w), so the maximum possible bitlength for any of these memory cells is $\log \max_i x[i]$. Similarly, at initialization, `inputlen` has value n , which corresponds to bitlength $\log n$. Summing these to achieve an upper bound on the maximum bitlength, we get $O(w + \log \max_i x[i] + \log n)$.

4. (reflection) Discuss the value (or lack thereof) that you think computational models, and the RAM and Word-RAM models in particular, have for computer science. All opinions are valid, as long as they show serious thought and are backed by specific justifications.

Quick note on grading: Good responses are usually about a paragraph, with something like 7 or 8 sentences. Most importantly, please make sure your answer is specific to this class and your experiences in it. If your answer could have been edited lightly to apply to another class at Harvard, points will be taken off.

Note: As with the previous psets, you may include your answer in your PDF submission, but the answer should ultimately go into a separate Gradescope submission form.

5. Once you're done with this problem set, please fill out [this survey](#) so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses!